

FindPath3D :

Laboratoire interactif de pathfinding en environnements voxel 3D

Chama EL KHEMSANI

15 septembre 2026

Résumé

FindPath3D est une application Python destinée à l'étude, la comparaison et la visualisation d'algorithmes de recherche de chemin dans des grilles tridimensionnelles. Le projet regroupe un moteur extensible (A*, Dijkstra, Theta*, JPS, TimeA*, etc.), des représentations spatiales (grille, octree), des contraintes physiques d'agent, des environnements dynamiques (portes temporelles, obstacles mobiles), des générateurs de cartes procéduraux et une interface graphique PyQt5 avec heatmap d'exploration et timeline de chemin. Ce rapport présente le contexte, le formalisme mathématique, la description détaillée des algorithmes implantés, l'architecture logicielle et les perspectives d'évolution.

Table des matières

1	Introduction	3
1.1	Contexte	3
1.2	Objectifs	3
1.3	Organisation du rapport	3
2	Modèle mathématique du problème	3
2.1	Espace discret 3D	3
2.2	Graphe de configuration	3
2.3	Coût d'une arête	3
2.4	Problème de plus court chemin	4
2.5	Contraintes physiques (filtre sur $\mathcal{N}$)	4
2.6	Environnement dynamique	4
3	Algorithmes de pathfinding : description et mathématiques	4
3.1	Dijkstra	4
3.2	A*	5
3.3	Weighted A* et Best-First	5
3.4	BFS	5
3.5	Heuristiques implantées	6
3.6	Theta* (any-angle)	6
3.7	Jump Point Search (JPS) — variante 3D	7
3.8	TimeA* (espace-temps)	7
3.9	D* et ARA* (aperçu)	7
3.10	Moteur génétique (exploratoire)	7
3.11	Synthèse comparative	8

4	Représentation spatiale	8
4.1	Grille	8
4.2	Octree	8
5	Architecture logicielle (aperçu)	8
6	Dynamique, génération et interface	8
7	Conclusion	8

1 Introduction

1.1 Contexte

La planification de trajectoires dans des espaces discrets tridimensionnels apparaît dans de nombreux domaines : robotique mobile, jeux vidéo, simulation, logistique, drones en intérieur. Les algorithmes de graphes (Dijkstra, A*, variantes *any-angle* ou *anytime*) constituent le socle classique de ces applications [1, 3].

1.2 Objectifs

FindPath3D propose un laboratoire unifié permettant de comparer plusieurs algorithmes sur les mêmes instances 3D, de faire varier la représentation spatiale, la physique de l'agent et la dynamique de l'environnement, et de mesurer temps, nœuds explorés, coût et optimalité.

1.3 Organisation du rapport

La section 2 formalise le problème. La section 3 décrit en détail les algorithmes et leur fondement mathématique. Les sections 4 à 6 présentent l'architecture et les modules complémentaires. La section 7 conclut.

2 Modèle mathématique du problème

2.1 Espace discret 3D

Soit une grille cubique de résolution $n \in \mathbb{N}^*$. L'ensemble des voxels est

$$\mathcal{V} = \{0, 1, \dots, n-1\}^3.$$

Un sous-ensemble $\mathcal{O} \subset \mathcal{V}$ désigne les **obstacles**. L'espace libre est $\mathcal{F} = \mathcal{V} \setminus \mathcal{O}$.

Définition 2.1 (Validité). Un point $p = (x, y, z) \in \mathbb{Z}^3$ est valide si $p \in \mathcal{V}$ et $p \notin \mathcal{O}$.

2.2 Graphe de configuration

On construit un graphe orienté (ou non) $G = (V, E)$ avec $V = \mathcal{F}$. Deux configurations $u, v \in \mathcal{F}$ sont voisines selon la **26-connexité** si

$$\max(|u_x - v_x|, |u_y - v_y|, |u_z - v_z|) = 1 \quad \text{et} \quad u \neq v.$$

L'ensemble des voisins de u est noté $\mathcal{N}(u)$.

2.3 Coût d'une arête

Le coût de déplacement de u vers v est la distance euclidienne :

$$c(u, v) = \|v - u\|_2 = \sqrt{(v_x - u_x)^2 + (v_y - u_y)^2 + (v_z - u_z)^2}. \quad (1)$$

Pour un déplacement unitaire axial, $c = 1$; pour une diagonale face, $c = \sqrt{2}$; pour une diagonale espace, $c = \sqrt{3}$.

2.4 Problème de plus court chemin

Étant donné un départ $s \in \mathcal{F}$ et un but $g \in \mathcal{F}$, un chemin est une suite

$$\pi = (p_0, p_1, \dots, p_L), \quad p_0 = s, \quad p_L = g, \quad p_{i+1} \in \mathcal{N}(p_i).$$

Son coût total est

$$C(\pi) = \sum_{i=0}^{L-1} c(p_i, p_{i+1}). \quad (2)$$

Le problème consiste à trouver $\pi^* \in \arg \min_{\pi} C(\pi)$ (ou déclarer l'absence de chemin).

Définition 2.2 (Distance optimale). On note $h^*(u)$ le coût minimal d'un chemin de u à g (avec $h^*(g) = 0$ et $h^*(u) = +\infty$ si g est inatteignable depuis u).

2.5 Contraintes physiques (filtre sur $\mathcal{N}$)

Soit un déplacement $u \rightarrow v$ avec $\Delta = v - u$. On définit :

$$d_{xy} = \sqrt{\Delta_x^2 + \Delta_y^2}, \quad (3)$$

$$\text{pente} = \begin{cases} \Delta_z / d_{xy} & \text{si } d_{xy} > 0 \text{ et } \Delta_z > 0, \\ +\infty & \text{si } d_{xy} = 0 \text{ et } \Delta_z > 0, \\ 0 & \text{sinon.} \end{cases} \quad (4)$$

Le mouvement est accepté si

$$\Delta_z \leq J_{\max} \quad \text{et} \quad \text{pente} \leq \tan(\theta_{\max}), \quad (5)$$

où $J_{\max}$ est la hauteur de saut maximale (en voxels) et $\theta_{\max}$ l'angle de pente maximal (ex. 45°). Optionnellement, un rayon de virage minimal rejette les changements de direction trop brusques.

Les voisins *physiquement admissibles* de u sont notés $\mathcal{N}_\phi(u) \subseteq \mathcal{N}(u)$.

2.6 Environnement dynamique

On enrichit la validité par le temps $t \in \mathbb{N}$. Une configuration (p, t) est libre si $p \in \mathcal{F}$ au temps t , en tenant compte de :

- portes Γ : ouvertes seulement sur des intervalles $\bigcup_k [a_k, b_k]$;
- zones de danger : bloquées si $\delta_0 + \gamma t > \tau$;
- obstacles mobiles suivant une trajectoire périodique.

Le graphe devient un graphe d'espace-temps dont les sommets sont des couples (p, t) .

3 Algorithmes de pathfinding : description et mathématiques

Sauf mention contraire, on travaille sur G avec voisins $\mathcal{N}$ ou $\mathcal{N}_\phi$, coût (1), départ s et but g .

3.1 Dijkstra

Dijkstra [2] maintient pour chaque nœud u une estimation $g(u)$ du coût depuis s , initialisée par $g(s) = 0$ et $g(u) = +\infty$ sinon. Une file de priorité extrait le nœud u de $g(u)$ minimal non encore *fermé*. Pour chaque $v \in \mathcal{N}(u)$:

$$\text{si } g(u) + c(u, v) < g(v) \quad \text{alors} \quad g(v) \leftarrow g(u) + c(u, v), \quad \text{parent}(v) \leftarrow u. \quad (6)$$

Proposition 3.1. Si tous les coûts c sont non négatifs, Dijkstra calcule $g(u) = d(s, u)$ pour tout u atteint, donc un plus court chemin vers g lorsque g est extrait.

Complexité typique avec tas binaire : $O((|V| + |E|) \log |V|)$.

3.2 A*

A* [1] enrichit la priorité par une heuristique $h : V \rightarrow \mathbb{R}_+$.

Définition 3.1 (Fonction f).

$$f(u) = g(u) + h(u),$$

où $g(u)$ est le coût du meilleur chemin connu de s à u , et $h(u)$ estime le reste jusqu'à g .

Définition 3.2 (Admissibilité). h est *admissible* si $\forall u, h(u) \leq h^*(u)$.

Définition 3.3 (Consistance (monotonie)). h est *consistante* si $\forall u, v \in E, h(u) \leq c(u, v) + h(v)$ et $h(g) = 0$.

Proposition 3.2 (Optimalité de A*). Si h est admissible et les coûts non négatifs, le premier chemin trouvé vers g est optimal. Si de plus h est consistante, chaque nœud n'est expandé qu'au plus une fois.

Schéma algorithmique.

Algorithm 1 A* (schéma)

```

1:  $g(s) \leftarrow 0; f(s) \leftarrow h(s); \text{Open} \leftarrow \{s\}; \text{Closed} \leftarrow \emptyset$ 
2: while  $\text{Open} \neq \emptyset$  do
3:    $u \leftarrow \arg \min_{x \in \text{Open}} f(x); \text{Open} \leftarrow \text{Open} \setminus \{u\}$ 
4:   if  $u = g$  then
5:     return chemin reconstruit via parent
6:   end if
7:    $\text{Closed} \leftarrow \text{Closed} \cup \{u\}$ 
8:   for all  $v \in \mathcal{N}(u)$  (ou  $\mathcal{N}_\phi(u)$ ) do
9:      $g' \leftarrow g(u) + c(u, v)$ 
10:    if  $v \notin \text{Closed}$  et  $g' < g(v)$  then
11:       $g(v) \leftarrow g'; f(v) \leftarrow g' + h(v); \text{parent}(v) \leftarrow u$ 
12:      ajouter ou mettre à jour  $v$  dans Open
13:    end if
14:  end for
15: end while
16: return échec

```

Dans FindPath3D, g est accumulé en coût euclidien réel le long des parents, et h est injectable (section 3.5).

3.3 Weighted A* et Best-First

Weighted A* utilise la priorité

$$f_w(u) = g(u) + w h(u), \quad w \geq 1. \quad (7)$$

Pour $w = 1$, on retrouve A*. Pour $w > 1$, l'algorithme est en général plus rapide mais le chemin peut être sous-optimal; sous certaines conditions, le coût est borné par $w \cdot C(\pi^*)$ [5].

Best-First (greedy) prend $f(u) = h(u)$ (équivalent à ignorer g). Aucune garantie d'optimalité; comportement souvent agressif vers le but.

Dijkstra se retrouve comme A* avec $h \equiv 0$.

3.4 BFS

Le parcours en largeur utilise une file FIFO. Si $c(u, v) = 1$ pour tout voisin (graphe non pondéré), BFS minimise le *nombre de pas*. Avec des coûts euclidiens variables (diagonales), BFS ne minimise plus $C(\pi)$.

3.5 Heuristiques implantées

Manhattan 3D.

$$h_M(u) = |g_x - u_x| + |g_y - u_y| + |g_z - u_z|. \quad (8)$$

Admissible pour un coût minimal d'arête ≥ 1 en déplacement axial uniquement ; en 26-connexité avec diagonales, elle peut sur-estimer relativement à c euclidien selon le barème exact — dans notre implantation, elle reste une heuristique simple de référence.

Euclidienne.

$$h_E(u) = \|g - u\|_2. \quad (9)$$

Admissible pour le coût (1) car la ligne droite minore tout chemin.

Octile 3D. En autorisant les diagonales, une forme classique combine les contributions axiales et diagonales. Une formule utilisée dans le projet s'écrit schématiquement :

$$h_O(u) = \Delta_x + \Delta_y + \Delta_z + (\sqrt{3} - 3) \min(\Delta_x, \Delta_y, \Delta_z), \quad (10)$$

avec $\Delta_x = |g_x - u_x|$, etc. (variantes équivalentes selon l'ordre des tris des composantes). L'idée est d'approcher le nombre de pas diagonaux vs axiaux.

Landmark (ALT simplifié). Soit un ensemble de landmarks $L = \{\ell_1, \dots, \ell_m\} \subset \mathcal{F}$. On précalcule, pour chaque $\ell \in L$, les distances $d(\ell, \cdot)$ par Dijkstra (ou équivalent) sur G .

Par inégalité triangulaire, pour tout u :

$$|d(\ell, u) - d(\ell, g)| \leq d(u, g) = h^*(u). \quad (11)$$

On pose

$$h_L(u) = \max_{\ell \in L} |d(\ell, u) - d(\ell, g)|. \quad (12)$$

Alors h_L est admissible. Plus les landmarks sont bien placés, plus h_L est informatif (moins de nœuds explorés).

Sélection adaptative. Une heuristique de topologie estime le taux d'occupation et la « congestion » locale pour choisir entre octile (espaces ouverts) et landmark (labyrinthes denses).

3.6 Theta* (any-angle)

Sur une grille, A* produit des chemins « en escalier ». Theta* [3] autorise un parent non adjacent si la **ligne de vue** est libre.

Définition 3.4 (Line-of-sight). $\text{LOS}(a, b)$ est vraie si le segment $[a, b]$ ne traverse aucun voxel obstacle (échantillonnage de type supercover / Bresenham 3D).

Lors de l'expansion de s vers un voisin n , soit $p = \text{parent}(s)$. Si $\text{LOS}(p, n)$, on tente la relaxation

$$g' = g(p) + c(p, n) \quad (13)$$

et l'on attache n à p plutôt qu'à s lorsque g' améliore $g(n)$. Sinon, on applique la règle A* classique $g(s) + c(s, n)$.

Remarque 3.1. Les chemins Theta* ont en général **moins de sommets** et un profil plus rectiligne, pour un coût total comparable ou meilleur en pratique sur grilles ouvertes. L'optimalité stricte dépend des hypothèses sur h et de la densité de la grille.

3.7 Jump Point Search (JPS) — variante 3D

JPS [4] réduit les nœuds expandés en *sautant* le long des directions libres tant qu'aucun **voisin forcé** n'apparaît.

Dans une direction $d \in \{-1, 0, 1\}^3 \setminus \{0\}$, un *jump* à partir de x avance $x \leftarrow x + d$ tant que :

- le voxel reste libre ;
- aucun voisin forcé (obstacle adjacent créant une dissymétrie) n'est détecté ;
- le but n'est pas atteint.

Le premier point où un voisin forcé apparaît, ou le but, devient un *jump point* successeur dans A^* .

Remarque 3.2. La théorie JPS est originellement 2D. L'implantation 3D de FindPath3D en est une **simplification pédagogique** : elle conserve l'idée des sauts directionnels et des voisins forcés, sans prétendre à l'équivalence formelle complète avec JPS 2D optimal.

3.8 TimeA* (espace-temps)

L'état devient $u = (p, t) \in \mathcal{F} \times \mathbb{N}$. Les transitions sont

$$(p, t) \rightarrow (p', t + 1), \quad p' \in \mathcal{N}_\phi(p) \cup \{p\} \quad (14)$$

(la présence de $p' = p$ modélise l'**attente**). Une transition est valide seulement si p' est libre au temps $t + 1$ (portes, danger, mobiles).

La fonction de coût accumule les $c(p, p')$ (avec $c(p, p) = 1$ pour l'attente, par convention de temps). L'heuristique ignore souvent le temps :

$$h(p, t) = h_{\text{spatial}}(p) \quad (15)$$

(admissible si attendre ne diminue pas le reste spatial minimal).

Un horizon T_{max} borne $t \leq t_0 + T_{\text{max}}$ pour garantir la terminaison.

Proposition 3.3 (Idée). Sous coûts non négatifs et heuristique admissible spatiale, TimeA* trouve un chemin faisable minimisant le coût choisi (distance ou temps selon le barème), s'il en existe un dans l'horizon.

3.9 D* et ARA* (aperçu)

D* (et D* Lite) [6] maintient une recherche orientée but et met à jour localement les coûts lorsque la carte change — pertinent pour la replanification.

ARA* (*Anytime Repairing A**) produit d'abord une solution sous-optimale contrôlée (facteur ε), puis améliore progressivement en diminuant ε , utile sous contrainte de temps de calcul.

3.10 Moteur génétique (exploratoire)

Une population de chemins (ou de séquences de mouvements) est évoluée par sélection, croisement et mutation. Aucune garantie d'optimalité ; l'intérêt est expérimental et pédagogique (métaheuristique vs méthodes exactes).

3.11 Synthèse comparative

Algorithme	Optimal [†]	Espace d'état	Usage typique
Dijkstra	Oui	V	Référence, $h = 0$
A*	Oui si h adm.	V	Usage général
Weighted A*	Non (borné)	V	Temps réel
Best-First	Non	V	Démo gloutonne
BFS	Oui (pas unitaires)	V	Graphe non pondéré
Theta*	Oui [‡]	V	Chemins fluides
JPS	Oui [‡]	V (succ. réduits)	Grilles régulières
TimeA*	Oui [†]	$V \times T$	Dynamique / temps

[†] Sous hypothèses standard (coûts ≥ 0 , h admissible le cas échéant).

[‡] Selon variantes et hypothèses de littérature ; implantation 3D simplifiée pour JPS.

4 Représentation spatiale

4.1 Grille

Stockage des obstacles dans un ensemble $\mathcal{O}$. Test d'appartenance en temps moyen $O(1)$.

4.2 Octree

Un nœud représente un pavé $[x_0, x_1) \times [y_0, y_1) \times [z_0, z_1)$. S'il contient trop d'obstacles et que la profondeur le permet, il est subdivisé en 8 octants. Les requêtes `IsObstacle` descendent l'arbre en $O(\log n)$ dans le pire cas typique. L'API publique reste identique à celle de la grille, ce qui rend le backend interchangeable.

5 Architecture logicielle (aperçu)

Le code sépare `core/` (moteurs, grille, physique, dynamique, heuristiques) et `ui/` (PyQt5, matplotlib). Le pattern Observer notifie exploration et chemin sans coupler le calcul à l'interface. Les tests `pytest` et les benchmarks assurent la non-régression et la mesure.

6 Dynamique, génération et interface

Dynamique. Portes temporelles, zones de danger, patrouilles et carburant via `DynamicMap` et `TimeA*`.

Générateurs. Neuf procédures (`random`, `maze`, `caves`, `rooms`, `pillars`, `perlin`, `bsp`, `city`, `floating`).

Interface. Paramètres d'expérience, heatmap d'exploration, timeline de chemin, dashboard et export CSV.

7 Conclusion

FindPath3D formalise le pathfinding 3D sur grille (coûts euclidiens, 26-connexité, physique optionnelle, extension temporelle) et implante une famille d'algorithmes allant de Dijkstra/A* à Theta*, JPS et TimeA*, avec heuristiques admissibles classiques et par landmarks. L'architecture modulaire et l'interface de mesure en font un support adapté à l'enseignement et à l'expérimentation.

Perspectives. Raffinement de JPS 3D, pattern databases, multi-agents, intégration robotique (ROS), campagnes de benchmarks systématiques.

Références

- [1] P. E. Hart, N. J. Nilsson, B. Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Trans. Systems Science and Cybernetics*, 4(2) :100–107, 1968.
- [2] E. W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1 :269–271, 1959.
- [3] A. Nash, K. Daniel, S. Koenig, A. Felner. Theta* : Any-angle path planning on grids. *Journal of Artificial Intelligence Research*, 39 :533–579, 2010.
- [4] D. Harabor, A. Grastien. Online graph pruning for pathfinding on grid maps. In *AAAI*, 2011.
- [5] I. Pohl. Heuristic search viewed as design space search. *Artificial Intelligence*, 1 :? ?, 1970. (Weighted heuristic search / ε -admissible variants.)
- [6] S. Koenig, M. Likhachev. D* Lite. In *AAAI/IAAI*, 2002 ; extended versions 2005.
- [7] D. Meagher. Geometric modeling using octree encoding. *Computer Graphics and Image Processing*, 19(2) :129–147, 1982.